TP 2 : ElasticSearch

idir.benouaret@epita.fr

1 Introduction et Contexte

Elasticsearch est un moteur de recherche et d'analyse distribué, conçu pour traiter rapidement de grandes quantités de données textuelles. Dans le cadre de ce projet de mini-Twitter, nous l'utiliserons pour gérer efficacement les services de recherche. Contrairement à MongoDB, qui sert au stockage principal des tweets, Elasticsearch sera utilisé dans votre projet pour l'indexation et la recherche en texte intégral. Il nous permettra d'indexer les posts en temps réel, d'effectuer des recherches rapides et précises par mots-clés, et d'optimiser le classement des résultats pour améliorer l'expérience utilisateur. Grâce à son modèle distribué et à son puissant moteur d'analyse, Elasticsearch garantit des performances élevées, même avec un volume massif de tweets, rendant ainsi la recherche fluide, rapide et pertinente.

L'objectif de ce TP est de découvrir Elastic Search et de vous familiariser avec son fonctionnement. Il fournit un moteur de recherche distribué et multi-entité à travers une interface REST. C'est un logiciel écrit en Java distribué sous licence Elastic.

Remarque : Si vous êtes en salle machine de l'école : sauter l'étape d'installation et utiliser la commande suivante pour lancer ElasticSearch

```
elasticsearch
```

Pour vérifier que le service est lancé, il suffit de taper la commande `curl -X GET "localhost:9200"` ou ouvrir un navigateur et vérifier l'url : `http://localhost:9200/`

Installation Pour l'installation, il est suggéré de suivre ce lien : `https://www.elastic.co/guide/en/elasticsearch/reference/8.0/targz.html`

- Pour des raisons de simplicité, veuillez configurer le ficher */elasticsearch-8.0.1/config/elasticsearch.yml* en mettant *xpack.security.enabled : false*

Pour démarrer le serveur ElasticSearch il suffit de se mettre sur le répertoire `elasticsearch-8.0.1` et taper la commande : `./bin/elasticsearch`

le démarrage du serveur peut prendre un peu de temps, pour vérifier, aller sur l'url : `http://localhost:9200/`

Il fonctionnera dès que vous obtenez une réponse sous forme de document JSON qui ressemble à ceci :


```
{
"name": "idir-pc",
"cluster_name": "elasticsearch",
"cluster_uuid": "BdTVRLD3RIG3gjAGKBh5PA",
"version": {
"number": "8.0.1",
"build_flavor": "default",
"build_type": "tar",
"build_hash": "801d9ccc7c2ee0f2cb121bbe22ab5af77a902372",
"build_date": "2022-02-24T13:55:40.601285296Z",
"build_snapshot": false,
"lucene_version": "9.0.0",
"minimum_wire_compatibility_version": "7.17.0",
"minimum_index_compatibility_version": "7.0.0"
},
"tagline": "You Know, for Search"
}
```


2 Présentation du travail avec ElasticSearch

Téléchargez le dossier tp2 se trouvant sur l'intra. Décompressez-le sur votre machine. Il crée un dossier tp2 pour faire le travail et contient :

- fichier de données
- scripts pour Elasticsearch
- et fichiers de soumission

Tout va se passer par http sur le port 9200 en localhost (variable HOST des fichiers bash).

Ouvrez le script test.sh à la racine du dossier `tp2` dans l'éditeur. Ce script lance des requêtes en bash. Ces requêtes sont groupées par fonction. Ces fonctions sont appelées l'une après l'autre à la fin du script.

Dans chaque fonction, vous remarquerez comment les titres en gras sont affichés, et comment des contenus JSON sont redirigés vers curl (redirection «MOT...MOT) : le mot clé JSON final doit être en début de ligne.

Lancez le script test.sh (**pensez à faire chmod +x test.sh**) si le fichier n'est pas exécutable.

Le script va créer un index dans Elasticsearch, appelé *tests*. dé-commentez les appels à schema et donnees, `recherche_url` et `recherche_dsl` sont des exemples de requête.

Ouvrez les URL suivants. **Et bien évidement en essayant pour chaque requête de la comprendre et comprendre le résultat qu'elle retourne** :

```
http://localhost:9200/tests/_search?q=valeur:7
http://localhost:9200/tests/_search?q=valeur:\-7
http://localhost:9200/tests/_search?q=valeur:"-7"
http://localhost:9200/tests/_search?q=valeur:>0
http://localhost:9200/tests/_search?q=nom:("a","c")
http://localhost:9200/tests/_search?q=nom:(-"a")
http://localhost:9200/tests/_search?q=-nom:"a"AND-valeur:7
http://localhost:9200/tests/_search?q=nom:["a"TO"b"]
```

Ce sont des recherches par URL. Notez la présence du \ devant le - (deuxième requête), c'est parce que le signe - est aussi un opérateur pour exclure des termes de la recherche. Comme ici, on cherche exactement la valeur -7 il faut la précéder d'un \. C'est aussi possible d'encadrer le -7 avec "-7"

Voyez comment agit le signe - pour bloquer des réponses, et comment employer les parenthèses pour grouper des conditions.

3 Exercices

Descendez dans le sous-dossier *ARBRES* du dosier tp2 et ouvrez le script arbres.sh dans un éditeur

Définition du schéma

La première opération consiste à définir le schéma de votre index. exécuter la fonction `schema()` du script arbres.sh

Avant de passer à la suite, assurez vous que le schéma a correctement été créé. Il suffit d'ouvrir l'url `http://localhost:9200/arbres` pour voir cela.

Injection des données

Maintenant que le schéma est prêt, il faut envoyer les données dans Elasticsearch. Commentez l'appel à `schema` et dé-commentez l'appel à `donnees`. La fonction `donnees` est déjà en place. Elle s'occupe d'envoyer le contenu du fichier `arbres.json` sur votre index.

Une fois le schéma défini et les données injectées sans erreurs, vous pouvez commenter les appels à ces fonctions à la fin du script.

La première recherche, `recherche_url` est exprimée en mode ligne. La deuxième requête `recherche_dsl` est écrite en DSL. Lire les transparents du cours pour avoir des éléments de syntaxe.

Avant de passer à la suite, assurez-vous qu'il y a des données dans votre index. Il suffit d'ouvrir l'URL `http://localhost:9200/arbres/_search?pretty`

Consignes pour la soumission de votre travail

Lisez attentivement les instructions suivantes avant de commencer les exercices

- Travailler/tester vos requêtes en éditant les fichiers qui se trouvent dans les sous-répertoires `ARBRES` et `FILMS`
- Chaque fichier bash correspond à une question. Par exemple, le fichier `exo2q3.sh` est pour travailler la question 3 de l'exercice 2.
- A la racine du dossier tp2, se trouvent les fichiers que vous devez soumettre et qui sont spécifié sur l'intra
- Pour les fichiers `.txt` : mettre uniquement le corps de la requête (voir exemple ci-dessous)
- Pour les fichiers `.json` : mettre uniquement le corps de la requête json (voir exemple ci-dessous)

Exemples

Si votre réponse à la question exo1q2 est :

```
function exo_1_2() {
Q='jenesaispastrop'
curl --get "${INDEX}/_search" --data-urlencode "q=${Q}" --data-urlencode "pretty"
}
exo_1_2
```

il faut soumettre mettre un fichier tp2/exo1q2.txt contenant uniquement le corps de votre requête, c'est à dire : `cat exo1q2.txt` donne

```
jenesaispastrop
```

Si votre réponse à la question exo2q2 est :

```
function exo_2_2() {
curljson -X GET "${INDEX}/_search?pretty" <<JSON
{
"query": {
    "match": {
    "attribut": "toto"
    }
  }
}
JSON
}
exo_2_2
```

il faut soumettre mettre un fichier tp2/exo2q2.json contenant uniquement le corps de votre requête, c'est à dire : `cat exo2q2.json` donne

```
{
"query": {
 "match": {
     "attribut": "toto"
       }
   }
}
```

Exercice I : Requêtes avec URI

RTFM : `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-query-string-query.html#query-string-syntax`

Q1 – Trouver les arbres du 5ème arrondissement

Q2 – Trouver les arbres dont le nom commun contient le mot "Erable" (sans accent)

Attention, il faut mettre un \devant chaque espace pour les requêtes avec URI.

Q3 – Trouver les arbres dont le nom commun est "Erable de Montpellier" (sans accent)

Q4 – Trouver les arbres dont la rue contient les mots "avenue" et "Matignon".

Q5 – Trouver les arbres dont la hauteur atteint ou dépasse 40 mètres

Q6 – Trouver les arbres du Bois de Vincennes plantés avant 1850 (l'année 1850 est comprise)

Q7 – Trouver les arbres du 19e siècle (année de plantation dans l'intervalle $[1801 - 1900]$)

Exercice II : Requêtes avec DSL

RTFM : `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl.html`

On passe maintenant à DSL qui permet beaucoup plus de choses, il s'agit de construire des requêtes en JSON.

Dans un premier temps, on va seulement retranscrire en DSL des requêtes précédentes, puis on verra les spécificités de DSL. Regardez la fonction `recherche_dsl` et comparez-la à `recherche_url`.

Q1 – Trouver des arbres dont la hauteur atteint ou dépasse 40 mètres

- ◇ `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-range-query.html`

Q2 – Trouver des arbres dont le nom commun contient « Erable » (sans accent).

- ◇ `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-match-query.html#query-dsl-match-query-short-ex`

Q3 – Trouver des arbres situé dans une rue qui contient Matignon et avenue

Il faut coder un opérateur booléen entre deux conditions "must"

- ◇ `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-bool-query.html`

Q4 – Trouver des arbres dont le nom commun contient le mot "Platane" mais pas le mot "commun" et qui sont dans le 12e arrondissement.

- ◇ `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-bool-query.html`

Exercice III : Agrégations

Il s'agit de regrouper certaines informations de plusieurs documents, par exemple compter, additionner, moyenner, etc.

Étudier la requête `agregation_dsl` et voir la doc :
`https://www.elastic.co/guide/en/elasticsearch/reference/8.11/search-aggregations.html`

Q1 – Afficher l'année de plantation du plus vieil arbre que vous nommez : `oldest_tree`

C'est à dire que votre requête doit renvoyer un résultat qui contient le champ suivant :

```
"aggregations" : {
  "oldest_tree" : {
    "value" : XXXX
    }
}
```

Q2 – Afficher la liste des lieux possédant au moins un arbre. Cette liste d'agrégation doit être nommée `liste_lieux`

Faire afficher la liste de tous les lieux, en un seul exemplaire chacun. Ce sont les valeurs distinctes du champ "adresse.lieu".

Votre requête doit renvoyer un résultat qui contient/ressemble au document suivant :

```
"aggregations" : {
  "liste_lieux" : {
  "doc_count_error_upper_bound" : 0,
  "sum_other_doc_count" : 27,
  "buckets" : [
    {
      "key" : "Bois de Boulogne",
      "doc_count" : 23
    },

    .
    .
    .
    {
    "key" : "Cimetiere du Pere Lachaise",
    "doc_count" : 2
    }
  ]
```

Exercice IV : Index sur des films

Passez maintenant sur le répertoire `FILMS` du dossier TP2.

1. Executer le script `films.sh` pour créer l'index *movies* et son schéma
2. Ensuite, tapez sur votre terminal la commande suivante :

```
curl -X PUT localhost:9200/_bulk -H "Content-Type: application/json" --data-binary
    @movies_elastic.json
```

La commande précédente envoie les documents JSON du fichier movies_elastic sur votre index.

Pour tester que c'est ok, tapez la commande :

- `curl -X GET http://localhost:9200/movies/_search`
- ou bien, vérifiez sur votre navigateur : `http://localhost:9200/movies/_search`

Répondez en DSL aux requêtes suivantes. A noter que tous les champs de l'index sont compris dans un champ `fields`

Q1 – Les films contenant "Star Wars" dans le titre

- `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-match-query-phrase.html`

Q2 – Les films contenant "Star Wars" dans le titre et où "George Lucas" est le directeur

Q3 – Les films où a joué Harrison Ford

Q4 – Les films où a joué Harrison Ford mais avec un plot contenant le mot 'Jones'

Q5 – Les films dirigés par James Cameron dont le rank est plus grand que 1000

Q6 – Les films dirigés par James Cameron avec un rating plus grand que 7 mais qui ne sont ni des films Fantasy ni des Thriller

Q7 – Le nombre de films pour chaque année. Nommez votre agrégation `nb_movies_each_year`

Votre requête doit renvoyer un résultat qui contient/ressemble au document suivant :

```
"aggregations" : {
  "nb_movies_each_year" : {
    "doc_count_error_upper_bound" : 0,
    "sum_other_doc_count" : 2265,
    "buckets" : [
      {
      "key" : 2013,
      "doc_count" : 456
        },
      .
      ,
       {
      "key" : 2014,
      "doc_count" : 157
      }
    ]
```

Q8 – La note moyenne des films d'actions. Nommez votre agrégation `mean_rating_action_movies`

Votre requête doit renvoyer un résultat qui contient :

```
"aggregations" : {
"mean_rating_action_movies" : {
"value" : xxxx.xxx
}
}
```